

Stereo Handoff — Worked Example

From operator CAPTURE to peer bounding box, computed by hand

jetson-tracking-perception • handoff branch

1 Setup

Two identical rectilinear cameras. Boresight (L) is above depression (R) by 100 mm along the Y axis. Depression is tilted 30° downward about its X axis. Both cameras have $f_x = f_y = 700$ px and principal point at $(c_x, c_y) = (640, 360)$ on a 1280×720 image. The target sits on a plane $d = 1000$ mm ahead of the boresight along its optical Z axis.

$$K_L = K_R = \begin{pmatrix} 700 & 0 & 640 \\ 0 & 700 & 360 \\ 0 & 0 & 1 \end{pmatrix}, \quad t = \begin{pmatrix} 0 \\ -100 \\ 0 \end{pmatrix} \text{ mm},$$
$$R = R_x(30^\circ) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0.866 & -0.500 \\ 0 & 0.500 & 0.866 \end{pmatrix}, \quad n = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \quad d = 1000$$

We will follow one target through the whole pipeline: operator captures on L at pixel $P_1 = (700, 550)$ with a 80×60 rectangle. We compute where the peer should look, what template it inherits, and what bounding box it draws.

2 Homography: where does the target appear in R?

The plane-induced homography maps a pixel in L to the corresponding pixel in R for any target on the assumed plane:

$$H = K_R \left(R - \frac{t n^\top}{d} \right) K_L^{-1}.$$

Two intermediate matrices to compute.

Outer product $t n^\top / d$.

$$\frac{t n^\top}{d} = \frac{1}{1000} \begin{pmatrix} 0 \\ -100 \\ 0 \end{pmatrix} \begin{pmatrix} 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & -0.1 \\ 0 & 0 & 0 \end{pmatrix}.$$

Bracket $R - t n^\top / d$.

$$R - \frac{t n^\top}{d} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0.866 & -0.400 \\ 0 & 0.500 & 0.866 \end{pmatrix}$$

(only the (2, 3) entry changed: $-0.500 + 0.100 = -0.400$).

Inverse of K_L .

$$K_L^{-1} = \begin{pmatrix} 0.001429 & 0 & -0.9143 \\ 0 & 0.001429 & -0.5143 \\ 0 & 0 & 1 \end{pmatrix}.$$

Full H . Multiplying $K_R \cdot (R - t n^\top / d) \cdot K_L^{-1}$:

$$H = \begin{pmatrix} 1.000 & 0.457 & -250.3 \\ 0 & 1.124 & -372.6 \\ 0 & 0.000714 & 0.609 \end{pmatrix}.$$

Apply to $P_1 = (700, 550)$.

$$H \begin{pmatrix} 700 \\ 550 \\ 1 \end{pmatrix} = \begin{pmatrix} 1.000 \cdot 700 + 0.457 \cdot 550 - 250.3 \\ 0 + 1.124 \cdot 550 - 372.6 \\ 0 + 0.000714 \cdot 550 + 0.609 \end{pmatrix} = \begin{pmatrix} 701.2 \\ 245.4 \\ 1.002 \end{pmatrix}.$$

Homogeneous divide:

$$P'_2 = \left(\frac{701.2}{1.002}, \frac{245.4}{1.002} \right) \approx (700.1, 245.0).$$

So the target, seen from R, appears at pixel (700, 245) — horizontally at the same column as L, vertically much higher (closer to the top of R's frame). This makes sense: R is tilted down, so a target on the ground plane appears in the upper portion of R.

3 Fundamental matrix: where MUST the target lie in R?

The fundamental matrix constrains the correspondence to a line, independent of depth:

$$F = K_R^{-\top} [t]_\times R K_L^{-1}.$$

Skew-symmetric matrix of t .

$$[t]_\times = \begin{pmatrix} 0 & 0 & -100 \\ 0 & 0 & 0 \\ 100 & 0 & 0 \end{pmatrix}.$$

Result after the three multiplications.

$$F = \begin{pmatrix} 0 & -1.02 \cdot 10^{-4} & -0.0870 \\ 0 & 0 & 0 \\ 0.1429 & 0.0653 & -35.75 \end{pmatrix}.$$

The zero middle row is fine — F has rank 2 by construction. Any pixel $P_1 = (u_L, v_L, 1)$ in L maps to a line $\ell_R = F P_1$ in R.

Apply to $P_1 = (700, 550)$.

$$a = 0 \cdot 700 + (-1.02 \cdot 10^{-4}) \cdot 550 + (-0.0870) \cdot 1 = -0.1431,$$

$$b = 0,$$

$$c = 0.1429 \cdot 700 + 0.0653 \cdot 550 + (-35.75) \cdot 1 = 100.20.$$

Normalise so $\sqrt{a^2 + b^2} = 1$. Divide through by $|a|$:

$$\ell_R: \quad -1.000 \cdot u + 0 \cdot v + 700.21 = 0 \quad \Longleftrightarrow \quad u = 700.21.$$

The epipolar line is a vertical line in R at column 700.21. Every valid correspondence of P_1 must lie on this line, at any v .

Sanity check

The homography said $P'_2 = (700.1, 245.0)$. Perpendicular distance from that point to the line $u = 700.21$ is $|700.1 - 700.21| = 0.11$ px. Homography and fundamental matrix agree. If they didn't, one of K , R , t , or d would be wrong.

4 Template bank: what does the peer inherit?

This section is the one to slow down for. The template bank is what lets handoff transfer *appearance memory*, not just *position*.

4.1 What is an embedding?

When the tracker looks at the target's image crop, it runs the crop through a small neural network (a *feature extractor*) that outputs a vector of numbers. That vector is the *embedding*. In our system it has $d = 1280$ numbers. The embedding is not the image; it's a compact numerical description of the image's *appearance*.

Two crops of the same object under similar conditions produce embeddings that are close to each other. Two crops of different objects produce embeddings that are far apart. "Close" and "far" are measured by *cosine similarity*.

Plain-English intuition

Think of the embedding as a fingerprint of what the crop looks like. Different crops of the same object have similar fingerprints; crops of different objects have very different fingerprints. We don't compare the raw images; we compare their fingerprints, which is much faster and much more robust to lighting, small shifts, or blur.

4.2 L2 normalisation

Every embedding is divided by its length so it becomes a unit vector:

$$\hat{e} = \frac{e}{\|e\|_2} = \frac{e}{\sqrt{\sum_k e_k^2}}.$$

This is done for one reason: after normalisation, the cosine similarity between two embeddings equals their dot product. No division needed at query time. Every stored template is already unit-length, so a query only needs to be normalised once when it arrives.

4.3 Cosine similarity, plain

For two unit vectors $\hat{a}$ and $\hat{b}$:

$$\text{sim}(\hat{a}, \hat{b}) = \hat{a} \cdot \hat{b} \quad \in \quad [-1, +1].$$

Read as: $+1$ = identical direction (same appearance), 0 = orthogonal (unrelated), -1 = opposite direction (rarely happens for real image features).

A 2D example makes it concrete:

$$\begin{array}{ll}
\hat{a} = (1, 0), & \hat{b} = (0.966, 0.259) \quad (15^\circ \text{ off from } \hat{a}) \\
\text{sim}(\hat{a}, \hat{b}) = 1 \cdot 0.966 + 0 \cdot 0.259 = 0.966 & \text{(very similar)} \\
\hat{c} = (0, 1) & \text{(orthogonal to } \hat{a}) \\
\text{sim}(\hat{a}, \hat{c}) = 0 & \text{(dissimilar)}
\end{array}$$

4.4 The bank is a gallery

Rather than storing one “current appearance”, the tracker keeps a small *gallery* of past appearances. Up to 12 entries, each recording what the target looked like at some earlier frame.

Why a gallery? A moving target changes appearance: different angles, lighting, self-occlusion. If we only kept the most recent view, we’d forget what the target looked like a second ago, and later reject a valid re-appearance because it doesn’t match the “current” template. With a gallery of past samples, the target only has to match *one* remembered look for the verifier to accept it.

4.5 Scoring a query against the bank

Given a new embedding $\hat{q}$, the bank score is the maximum similarity over all stored templates:

$$S(\hat{q}; \mathcal{B}) = \max_{i=1\dots n} \hat{q} \cdot \hat{e}_i.$$

The MAX. Not the average. Not the sum. The single best match. This is the answer to “does the target still look like *something* I remember?”

Plain-English intuition

If the target is at a weird angle right now and only matches template number 7 (which happened to see it at a similar angle a few seconds ago), that’s enough. Templates 1–6 and 8–12 can score 0.2 and it doesn’t matter — the max picks up the 0.85 from template 7.

4.6 Growing the bank — three rules

The bank starts empty. On operator CAPTURE:

1. Extract crop around the operator’s rectangle, embed, normalise — that’s the seed template $\hat{e}_0$.
2. Store as entry 1.

During TRACKING, on each verified frame:

1. If the current crop’s score against the bank is above $\tau_{\text{accept}} = 0.60$, the tracker is confidently seeing the target.
2. If enough frames have passed since the last add ($\Delta_{\text{min}} = 30$ frames), add this new normalised embedding to the bank.
3. If the bank is now over capacity ($C = 12$ entries), evict the *oldest non-seed* entry. The seed template stays forever.

Plain-English intuition

Two rules keep the bank useful:

Spacing. Frames 1–30 all show the target basically the same way; storing every frame would fill the bank with near-duplicates. The 30-frame gap ensures we capture appearance *changes over time*, not *copies*. Diversity over redundancy.

Keep the seed. Later entries can drift (small verification errors accumulate). The seed is the operator’s ground truth — it anchors the memory to the original object no matter how much drift happens in later frames.

4.7 Using the bank to reject distractors

Every few frames (default: every 6 in TRACKING), the tracker verifies its current crop against the bank:

- $S < \tau_{\text{veto}} = 0.45$: current crop does NOT match the target. Vote “mismatch” (increment `vetoStreak`). Three consecutive votes force the tracker into LOST.
- $S \geq \tau_{\text{accept}} = 0.60$: current crop matches. Trust it. Possibly grow the bank if spacing allows.
- In between: abstain. No vote, no add.

This is how the tracker resists locking onto the wrong thing when a similar-looking distractor drifts into the search window.

4.8 Worked numeric example ($d = 4$ for readability)

Real embeddings have $d = 1280$; nothing about the math changes if we use $d = 4$ so we can read the numbers.

Frame 0 — CAPTURE. Crop yields raw embedding $e_0 = (1, 1, 1, 1)$. Length $\|e_0\| = \sqrt{4} = 2$. Normalise:

$$\hat{e}_0 = \frac{1}{2}(1, 1, 1, 1) = (0.500, 0.500, 0.500, 0.500).$$

Bank state: $\mathcal{B} = \{\hat{e}_0\}$, $f_{\text{lastAdd}} = 0$.

Frame 30 — add? Current embedding $e_1 = (1.1, 0.9, 1.05, 0.95)$. Length $\|e_1\| = \sqrt{4.025} \approx 2.006$. Normalise:

$$\hat{e}_1 = (0.548, 0.449, 0.523, 0.474).$$

Bank score:

$$S(\hat{e}_1; \mathcal{B}) = \hat{e}_1 \cdot \hat{e}_0 = 0.548(0.5) + 0.449(0.5) + 0.523(0.5) + 0.474(0.5) = 0.997.$$

Score $\geq \tau_{\text{accept}}$ and spacing $\Delta_{\text{min}} = 30$ satisfied. *Add.* Bank $|\mathcal{B}| = 2$.

Frame 60 — add? $e_2 = (1.2, 0.8, 1.0, 1.0)$, $\|e_2\| \approx 2.020$, $\hat{e}_2 = (0.594, 0.396, 0.495, 0.495)$.

$$S = \max\{0.990 \text{ (vs } \hat{e}_0), 0.998 \text{ (vs } \hat{e}_1)\} = 0.998.$$

Above accept, spacing OK. *Add.* Bank $|\mathcal{B}| = 3$.

Frame 65 — abstain from adding. A query $\hat{q} = (0.449, 0.499, 0.549, 0.499)$ arrives.

$$S = \max\{0.998, 0.988, 0.973\} = 0.998.$$

Verifier accepts (target still looks right). But only 5 frames since $f_{\text{lastAdd}} = 60$; spacing rule blocks the add. Bank unchanged.

Frame 100 — distractor. A very different-looking crop gives $q_{\text{dist}} = (1, -1, 1, -1)$, normalised to $\hat{q}_{\text{dist}} = (0.5, -0.5, 0.5, -0.5)$.

$$S(\hat{q}_{\text{dist}}; \mathcal{B}) = 0.5 - 0.5 + 0.5 - 0.5 = 0.$$

Below $\tau_{\text{veto}} = 0.45$. Vote “mismatch”. Three of these in a row and the tracker switches itself to LOST rather than commit to the wrong object.

4.9 Handoff transfer: serialising the whole bank

At the auto-seed moment, the source’s entire bank is serialised to a byte buffer and imported into the peer. The peer’s tracker uses the source’s bank as its own starting gallery — so peer inherits “twelve past appearances of the target” rather than starting with just its own one-frame view.

Byte size. Header 16 bytes (magic + version + dim + count), then each entry is 4 bytes for the frame counter plus $4d$ bytes for the d -dim float embedding.

$$\text{size}(\mathcal{B}) = 16 + n \cdot (4 + 4d).$$

For a full bank with $d = 1280$ and $n = 12$: $16 + 12 \cdot 5124 = 61,504$ bytes ≈ 60 KB. Transferred in-process (same address space, no network), so the cost is a memcpy.

Plain-English intuition

The picture to hold: when the peer camera captures for the first time, its bank has just one template (the one crop it just took). One frame of memory. That template might mis-verify future frames because it’s a single sample. After the import, the peer’s bank suddenly has all 12 of the source’s templates too — 12 frames’ worth of memory across scales, lighting, and orientations. From frame 2 onward the peer starts adding its own high-confidence samples too, mixing L’s and R’s viewpoints.

5 Bounding box: what size does the peer draw?

Source rect 80×60 px at $f_{x,L} = 700$. Peer rect uses focal ratios:

$$w_R = w_L \cdot \frac{f_{x,R}}{f_{x,L}}, \quad h_R = h_L \cdot \frac{f_{y,R}}{f_{y,L}}.$$

For our identical cameras ($f_{x,R} = 700$):

$$w_R = 80 \cdot \frac{700}{700} = 80, \quad h_R = 60 \cdot \frac{700}{700} = 60.$$

Peer draws an 80×60 rectangle centred at $P'_2 = (700, 245)$.

Different-lens example: if depression had $f_{x,R} = 900$ (longer focal), $w_R = 80 \cdot 900/700 \approx 103$ px — same physical object subtends more pixels because the peer has a narrower FOV. The formula is a first-order approximation, exact when the target sits at the assumed depth d and off by a few percent when it doesn’t. Good enough — the peer’s own correlation filter refines the box from the next frame onward.

6 The full handoff, frame by frame

Frame	Event	State change
0	Operator CAPTURE on L at (700, 550), rect 80×60 .	L: TRACKING, det = 1.00. Bank L: $\{\hat{e}_0\}$. Primary = L.
1–29	L tracks. R stays FREE.	L bank still $\{\hat{e}_0\}$ (spacing gate). Seed check every frame; target may not be in overlap yet.
30	L adds $\hat{e}_1$ to bank.	$ \mathcal{B}_L = 2$.
60	L adds $\hat{e}_2$; target enters overlap.	$ \mathcal{B}_L = 3$. Seed condition met (L TRACKING, det > 0.4, R FREE).
60 ⁺	Compute $P'_2 = HP_1 = (700, 245)$; line $\ell_R : u = 700.21$.	Both inside R frame. Fire seed.
60 ⁺	Serialise L bank (3 templates, $d = 4$) $\rightarrow$ 64 bytes.	Header $16 + 3 \cdot (4 + 16) = 64$. Real $d = 1280$: ~ 60 KB.
60 ⁺	CAPTURE on R at (700, 245).	R: TRACKING, bank R = $\{\hat{r}_0\}$ (one crop from R's view).
60 ⁺	Import L bank into R.	R bank overwritten to $\{\hat{e}_0, \hat{e}_1, \hat{e}_2\}$.
60 ⁺	$R.setEpipolarConstraint(a = -1, b = 0, c = 700.21, halfW = 32, TTL = 60)$.	R's peak selection prefers peaks within 32 px of $u = 700.21$ for 60 frames.
61	R's first processFrame after seed.	Correlation searches around (700, 245); if primary peak lands off the line, R switches to closest-to-line secondary peak.
61–120	Both cameras TRACKING. UART primary stays L. R grows its own high-confidence samples via maybeAdd $\rightarrow$ mixed L+R history.	
~ 150	L drops to LOST at the FOV edge; R is TRACKING with det ≥ 0.5 .	Auto primary-flip: R takes UART. FC keeps receiving angles.

7 Sanity checks the reader can do at the desk

Sanity check

Homography seed vs. epipolar line. Perpendicular distance from HP_1 to $\ell_R = FP_1$ should be $\leq 1\text{--}2$ px for a well-calibrated rig. Larger $\Rightarrow K$, R , or t inconsistent. In our worked example the distance was 0.11 px.

Sanity check

Seed inside peer frame. If (u, v) falls outside $[0, W) \times [0, H)$, the target isn't in the overlap yet — code logs `[HANDOFF-*] projected pixel (...) outside peer frame` and skips the seed. Wait for the object to descend further into R's FOV, or re-check the plane depth assumption.

Sanity check

Bank score after handoff. If R's first ten frames after import score $S < \tau_{\text{accept}}$, the transferred templates are a bad fit for R's viewpoint — most often exposure or colour balance differs between the two cameras. Fix at the camera driver level, not by lowering the threshold.

Sanity check

Epipolar-line direction check. A pure vertical baseline ($t_x = t_z = 0$) with rotation about X (tilt) produces vertical epipolar lines in the peer, as we got here ($u = \text{const}$). If your lines come out horizontal or diagonal, your baseline or rotation is misconfigured.